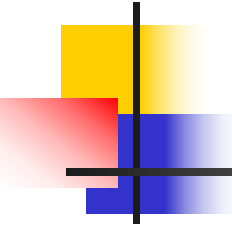


第3章 指令系统及汇编语言程序设计-1

- 3.1 概述
- 3.2 寻址方式
- 3.3 汇编语言的编程格式
- 3.4 指令系统
 - 3.4.1 数据传送类指令



3.1 概述

基本概念

- **计算机程序**：一系列指令的有序集合
- **指令**：计算机能够识别和执行的操作命令
- **指令系统（指令集）**：计算机能执行的全部操作命令，表征了计算机的基本功能

计算机的 **CPU** 不同，指令系统也不同。

指令系统还规定了每条指令的表示格式。

指令有两种书写格式：**机器指令**和**符号指令**。

3.1.1 机器指令格式

机器指令一般由操作码和操作数组成

操作码：说明指令操作的性质与所完成的功能，常用 **OP** 表示

操作数：给出了参与操作的对象

- 有些指令不需要操作数，通常的指令都有一个或两个操作数，也有个别指令有3个甚至4个操作数

8086/8088 指令编码由 1~6 个字节组成

操作码(1 字节)	寻址方式(无或 1 字节)	操作数(无或 1~4 字节)
-----------	---------------	----------------

例如：加法指令： **ADD AX, [BX+DI+2000H]**

其指令编码是： **03H、81H、00H、20H。**



3.1.1 符号指令格式

[标号:] 操作码助记符 操作数助记符 [;注释]

存放的地址

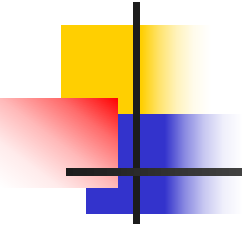
完成的操作

操作数个数、位置及数值

双操作数指令 **MOV AX, 1234H ;立即数送AX寄存器**

目的操作数 源操作数

- 用机器指令编写的程序称为**目标程序**
- 用符号指令编写的程序称为**符号程序**或**汇编语言源程序**
- 汇编语言源程序要经过**汇编程序**将其编译和链接后才能生成 **CPU** 能识别和执行的**目标程序**



3.2 寻址方式

寻址方式：说明指令中如何提供操作数或提供操作数存放地址的方法，指令中的操作数有4类寻址方式

- ◆ 立即寻址方式 `MOV AX, 2000H` ;操作数就在指令中
- ◆ 寄存器寻址方式 `MOV AX, BX` ;操作数在CPU的寄存器中
- ◆ 存储器寻址方式 类型多样，操作数在存储器中
- ◆ I/O 端口寻址方式 `IN AL, DX` ;操作数在 I/O 接口端口中

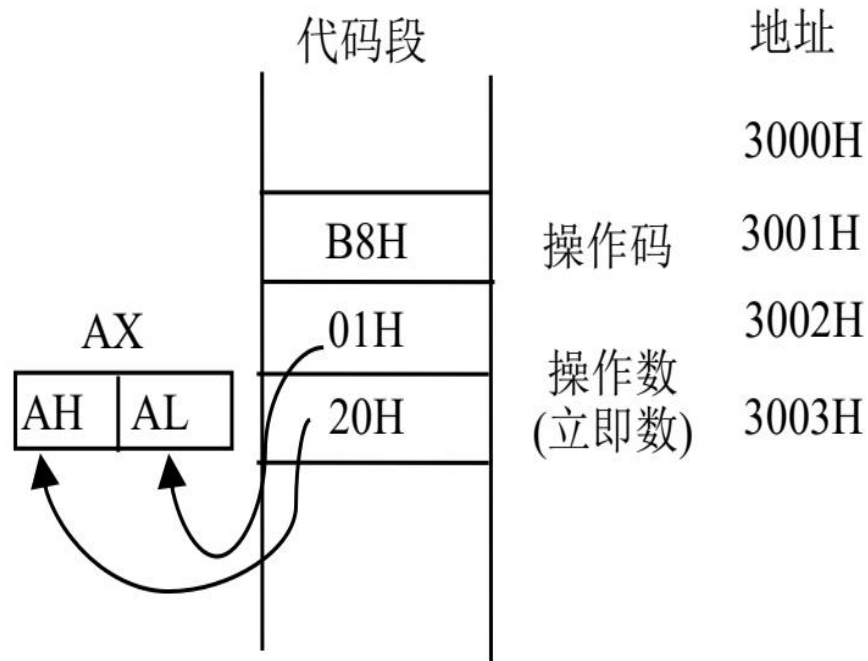
3.2.1 立即寻址方式

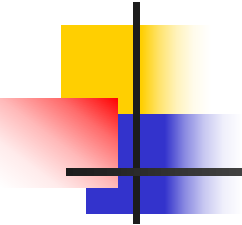
- ◆ 指令中的**操作数直接存放在机器代码**中，紧跟在操作码之后（操作数作为指令的一部分存放在操作码之后的主存单元中）

MOV AX, 2001H ;机器代码：**B8 01 20**

作用：常用来给寄存器或存储单元赋值，多以常量形式出现。

注意：只有源操作数才允许用立即数。





3.2.2 寄存器寻址方式

- ◆ 操作数存放在**CPU**的内部寄存器中
 - 8个通用寄存器和4个段寄存器
 - 寄存器名表示其内容（操作数）

MOV AX, BX ; AX←BX

MOV AX, 2345H

； **2345H** 送 **AX**，源操作数为立即寻址，目的操作数为寄存器寻址

3.2.3 存储器寻址方式

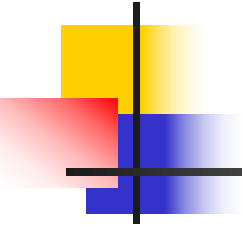
- ◆ 操作数在**主存储器**中，用主存地址表示
- ◆ 程序设计时，采用逻辑地址表示主存地址
 - 段地址在默认的或用段超越前缀指定的段寄存器中
 - 指令中只需给出操作数的偏移地址（有效地址**EA**）

逻辑地址 = 段基址 SBA: 有效地址 EA

- ◆ **SBA** 是逻辑段的段基地址，由各段寄存器给定。
- ◆ **EA** 是存储单元与段基址间的字节距离，即段内偏移地址

物理地址 = 段基址 × 16 + 有效地址

存储器寻址方式的标志是**符号指令中操作数有中括号 []**



3.2.3 存储器寻址方式

(1) 直接寻址

- ◆ 有效地址在指令中直接给出
- ◆ 默认的段地址在**DS**段寄存器，可使用段超越前缀改变
- ◆ 用中括号包含有效地址，表达存储单元的内容

3.2.3 存储器寻址方式

(1) 直接寻址

例：MOV AX, [2000H]

有效地址

• 设 DS=3000H,

物理地址=DS*16+2000H=32000H

• AX=0100H

• [2000H] = DS:[2000H]

• 段超越：

ES:[2000H], SS:[2000H]

内存(H)

有效地址

操作码

00H

20H

代码段

32000H

32001H

00H

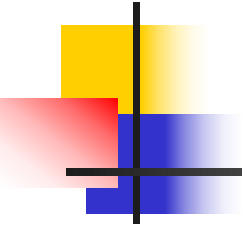
01H

04H

09H

10H

数据段



3.2.3 存储器寻址方式

(2) 寄存器间接寻址

- ◆ 有效地址存放在**基址寄存器BX、BP**或**变址寄存器SI、DI**中，注意不可以使用其他寄存器。
- ◆ 段地址对应**BX/SI/DI**寄存器**默认是DS**，对应**BP**寄存器**默认是SS**；可用段超越前缀改变

(2) 寄存器间接寻址

例：MOV AX, [BX];
间接地址

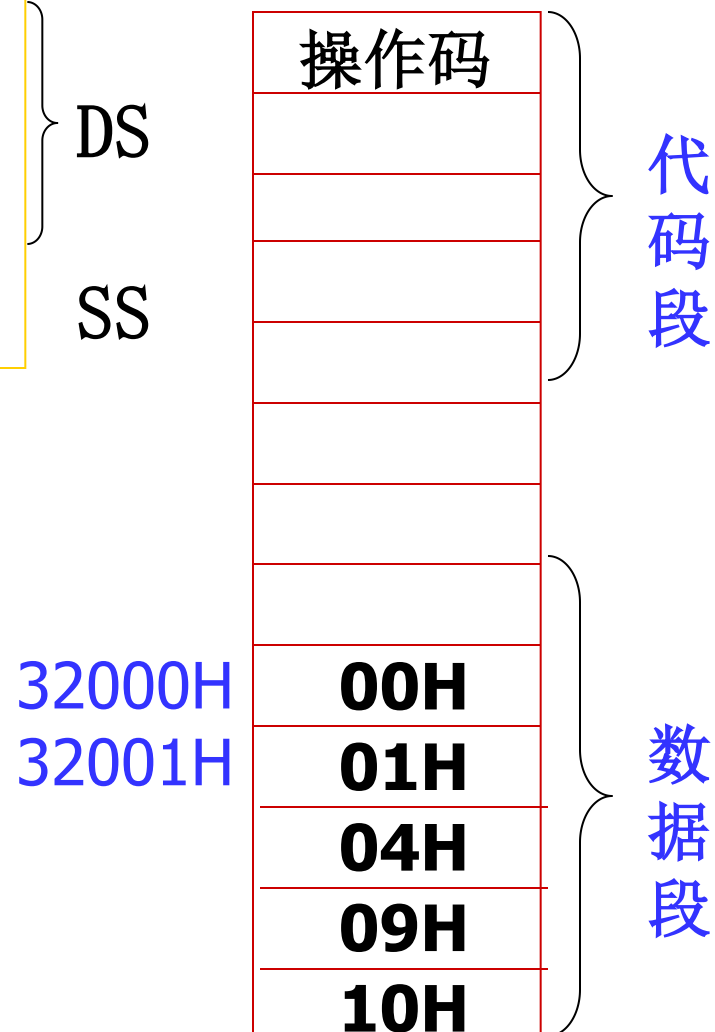
BX	DS
SI	
DI	
BP	SS

- 设 DS=3000H, BX=2000H,
DS*16+BX=32000H
- AX= 0100H

思考指令功能？

MOV BX, ES: [DI]

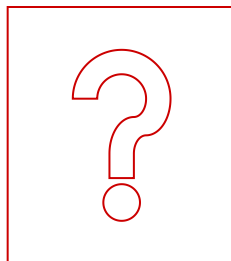
内存(H)



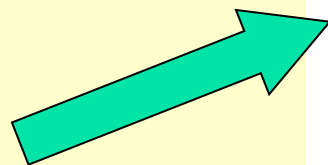
(2) 寄存器间接寻址

求2000H开始的100个数的和

```
MOV  AL, 0
ADD  AL, [2000H]
ADD  AL, [2001H]
...
```



```
MOV  AL, 0
MOV  BX, 2000H
ADD  AL, [BX]
INC  BX
```



LOOP NEXT

```
NEXT: ADD  AL, [BX]
        INC  BX
        DEC  CX
        JNZ  NEXT
```

(3) 寄存器相对寻址

EA = BX/BP/SI/DI + 8 / 16 位位移量

例： MOV AL, [BX+03H] ;

有效地址

BX } DS 位移量
SI }
DI }
BP } SS

• 设 DS=3000H, BX=2000H,
DS*16+BX+03H=32003H

• 结果： AL=09H

; BX 为数组的首地址

MOV AL, [BX+N]

; 功能： 指向数组中的第N个数

内存(H)

操作码

03H

00H

01H

04H

09H

10H

19H

代码段

数据段

(4) 基址变址寻址

EA = BX/BP + SI/DI

例: MOV AX, [BX+SI];

有效地址

BX+SI

BX+DI

BP+SI

BP+DI

DS

SS

内存(H)

操作码

代码段

• 设 DS=3000H, BX=1000H, SI=1000H

DS*16+BX+SI=32000H

• 结果: AX=0100H

32000H

32001H

00H

01H

04H

09H

10H

数据段

指向数组中的第SI个数

MOV AL, [BX+SI]

(5) 相对基址变址寻址

内存(H)

$EA = BX/BP + SI/DI + 8/16$ 位位移量

例：MOV AX, [BX+SI+0200H]；

有效地址

BX+SI

BX+DI

BP+SI

BP+DI

位移量

DS

SS

• 设 DS=3000H

BX=1000H

SI=0500H

$DS * 16 + BX + SI + 0200H = 31700H$

31700H

31701H

• 结果：AX=1234H

• 用于二维数组操作，N为数组起始地址

MOV AL, [BX+SI+N]

操作码

00H

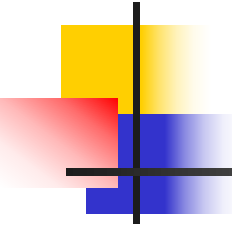
02H

34H

12H

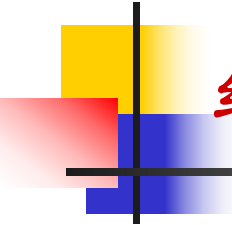
代码段

数据段



(6) 相对寻址的多种写法

例：MOV AX, [BX+SI+0200H] ;
MOV AX, [BX+SI]+0200H ;
MOV AX, [BX][SI]+0200H ;
MOV AX, 0200H[BX+SI] ;
MOV AX, 0200H[BX][SI] ;
MOV AX, 0200H+[BX][SI] ;
MOV AX, 0200H+[BX+SI] ;
MOV AX, [BX+0200H][SI] ;
.....



练习:指出下列指令中源操作数的寻址方式

- | | | |
|-----------|-----------------|----------|
| (1) MOV | SI, 200 | 立即寻址 |
| (2) MOV | AL, [2000H] | 直接寻址 |
| (3) MOV | CX, DATA[SI] | 寄存器相对寻址 |
| (4) ADD | AX, [BX+DI] | 基址变址寻址 |
| (5) AND | AX, BX | 寄存器寻址 |
| (6) MOV | AX, DATA[BP+SI] | 相对基址变址寻址 |
| (7) PUSHF | | 隐含寻址 |
| (8) MOV | AX, ES:[BX] | 寄存器间接寻址 |
| (9) JMP | FAR PTR PROCS_1 | 段间直接转移 |

练习:指出AX寄存器中的内容

3.5 已知**DS=2000H**, **BX= 0100H**, **SI=0002H**和如图存储器存储情况, 试说明下列指令执行后, **AX**寄存器中的内容。

(1) **MOV AX, 1200H**

20100H

12H

20101H

34H

20102H

56H

(3) **MOV AX, [1200H]**

20103H

78H

...

(5) **MOV AX, 1100H[BX]**

21200H

2AH

21201H

4CH

21202H

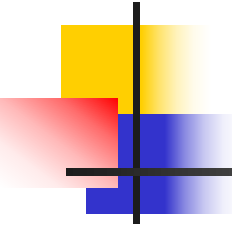
B7H

(7) **MOV AX, [1100H+BX+SI]**

21203H

65H

...



3.3 汇编语言的编程格式

- ◆ 汇编语言是一种以处理器指令系统为基础的低级程序设计语言，它采用助记符表达指令操作码，采用标识符表示指令操作数
- ◆ 利用汇编语言编写程序的主要优点：
 - 可以直接、有效地控制计算机硬件
 - 容易创建代码序列短小、运行快速的可执行程序
- ◆ 汇编语言不足，功能有限、编程繁难、依赖处理器指令

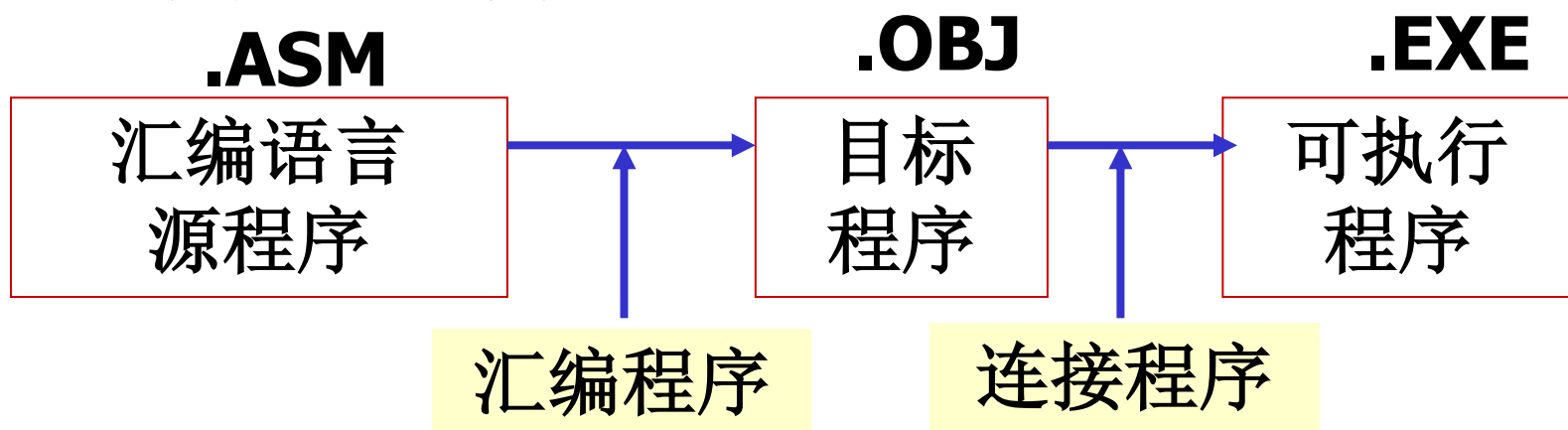
3.3.1 汇编语言程序结构

1. 汇编语言完整的源程序结构

stack	SEGMENT		}	堆栈段
	DB	100 DUP(0)		
stack	ENDS		}	数据段 存放数据、变量
data	SEGMENT	<数据、变量在此定义>		
data	ENDS		}	代码段 存放程序指令
code	SEGMENT	ASSUME CS:code, DS:data, ES:data, SS:stack		
start:	MOV	AX, data	}	
	MOV	DS, AX ;给 DS 赋值为数据段段基址		
	MOV	ES, AX ;给 ES 赋值为附加段段基址		
		<此处加入你自己的程序段>		
	MOV	AX, 4C00H		
	INT	21H		
code	ENDS			
	END	start		

3.3.1 汇编语言程序结构

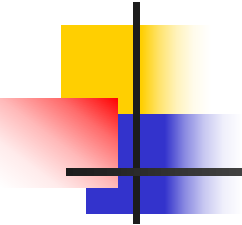
2. 汇编程序的运行步骤



- (1) 输入源程序
- (2) 保存
- (3) 编译、生成可执行文件

运行?





3.3.1 汇编语言程序结构

(4) 调试

反汇编查看汇编程序: **U** (一次**32**字节)

运行:

连续运行命令: **G=0000 0017** (程序开始,结束位置)

单步运行命令: **T**

查看结果:

在寄存器内: **R**

在内存 : **D** 段地址:偏移地址

退出调试: **Q**

3.3.1 汇编语言程序结构

DEBUG 命令

名称	解释	格式
a (Assemble)	逐行汇编	a [address]
c (Compare)	比较两内存块	c range address
d(Dump)	内存16进制显示	d [address]或 d [range]
e (Enter)	修改内存字节	e address [list]
f (fin)	预置一段内存	f range list
g (Go)	执行程序	g [=address][address...]
h (Hexavithmetic)	制算术运算	h value value
i (Input)	从指定端口地址输入	i portaddress
l (Load)	读盘	l [address [driver sector>
m (Move)	内存块传送	m range address
n (Name)	置文件名	n filespec [filespec...]
o (Output)	从指定端口地址输出	o portaddress byte
q (Quit)	结束	q
r (Register)	显示和修改寄存器	r [register name]
s (Search)	查找字节串	s range list
t (Trace)	跟踪执行	t [=address] [value]
u (Unassemble)	反汇编	u [address]或range
w (Write)	存盘	w [address[driver sector secnum>
?	联机帮助	?

3.3.2 汇编语言语句

➤ 指令语句

[标号:] 指令助记符 [操作数[,操作数]] [;注释]

例 **START: MOV AX, DATA**

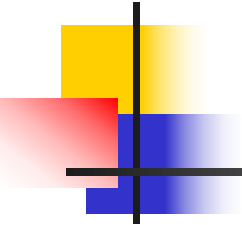
标号: 该指令的符号地址, 第一个字符为字母

➤ 指示性/伪指令语句

[名字] 伪指令助记符 [操作数[,操作数,...]] [;注释]

例 **DATA1 DB 11H, 22H, 33H, 44H**

标号和名字的起名规则: 必须以字母开头, 后跟字母、数字 (0, ..., 9) 及特殊符号 (? , @, _, \$等) 等组成, 长度为 **1~31** 个字符, 不能是属于系统的专用保留字。



3.3.3 汇编语言的操作数

1. 寄存器

- 8位：字节型寄存器，有AH/AL/BH/BL/CH/CL/DH/DI
- 16位：字型寄存器，有AX/BX/CX/DX/SI/DI/BP/SP等

2. 常量

- 运行期间不会改变的固定值
 - (1) **数字常量**：各种进制形式的数值
 - 如**100 (D) / 01100100B / 255Q / 64H / 0A0H**
 - (2) **字符串常量**，单双引号括起来的一个或多个字符
 - 例：'A', 'ABCD' ;对应字符的**ASCII**码

3.3.3 汇编语言的操作数

3. 存储器操作数

通常是一个标识符，有标号（**Label**）和变量（**Variable**）两种：

- **标号**：代表存放该指令的存储单元的符号地址，其后须加冒号，通常作为转移类指令、过程调用指令和循环控制类指令的操作数；
- **变量**：存储器中某个数据区的名字，内容可改变，变量值可改变。

◆ 标号和变量的属性

- **段地址属性**：标号所在代码段、变量所在数据段/附加段的段基地址；
- **偏移地址属性**：标号或变量所在段的段首到定义该标号或变量的地址之间的字节数，即**偏移地址**，为16位无符号数。
- **类型属性**：标号的类型有**NEAR**（近标号，段内被引用、16位）、**FAR**（远标号，可在其它段被引用、32位）两种；

变量类型：**BYTE/WORD/DWORD/QWORD/TBYTE**

◆ **type** : 获得类型属性

3.3.3 汇编语言的操作数

3. 存储器操作数

- **BYTE**（字节）、**WORD**（字）和**DWORD**（双字），
TYPE = 1、2和4

例： **D1 DB 1, 2, 3, 4, 5** ； 定义变量**D1**为字节类型
MOV AL, TYPE D1 ;AL=1

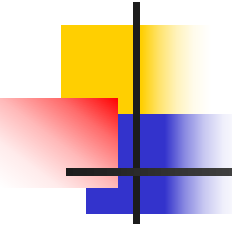
- **地址操作符**取得名字或标号的段地址和偏移地址

[]	将括起的表达式作为存储器地址指针
\$	当前偏移地址
:	段前缀，采用指定的段地址寄存器
OFFSET 名字/标号	返回名字或标号的偏移地址
SEG 名字/标号	返回名字或标号的段地址

3.3.3 汇编语言的操作数

4. 表达式

- ◆ 汇编程序在汇编过程中计算表达式并得到一个**确定的数值**，组成表达式的各部分必须在汇编时就有确定的值。
- **算术运算符**: +, -, *, /, **MOD** , 如 **MOV AL, 8+6**
- **逻辑运算符**: **AND, OR, NOT, XOR**, 如 **MOV BL, NOT 0FFH**
- **关系运算符**: **EQ, NE, LT, GT, LE, GE**, 如 **MOV BX, 6 LT 5**
 - ✓ 参与关系运算的必须是两个数值
 - ✓ 关系不成立，结果为全**0**；否则为全**1**
- **分析运算符和合成运算符**: 下一小节介绍
- **其它运算符**
 - ✓ 方括号[], 表示存储器操作数，方括号内容表示偏移地址；
 - ✓ 段超越运算符**:"**，跟在某个段寄存器名（**DS/ES/SS**）后，表示重设段基址，指定一个存储器操作数的**段属性**，
如 **MOV AX, ES:[DI]**



3.3.4 伪指令

- ◆ **指令语句**：程序运行时由CPU执行，每条指令对应CPU一种操作，如加/减法、数据传送等；汇编后产生目标代码。
- ◆ **伪指令**：在汇编过程中由汇编程序执行，提供汇编所需信息，如程序的开始与结束、数据区定义及原始数据、存储区的分配等；汇编后不产生目标代码。

3.3.4 伪指令

1. 符号定义伪指令 用于定义符号常量，不占内存存储空间

- 符号定义伪指令有“等值**EQU**”和“等号=”：

符号名 **EQU** 数值表达式

；给某个表达式赋予一个名字，在源程序中用该名字代替表达式；

符号名 **EQU** <字符串>

符号名 = 数值表达式 ；“等号=”不能定义字符串

- EQU**不能重复定义符号名，但“=”允许重复赋值。

例如：X = 7 ；等效于：X **EQU** 7

X = X+5 ；“X **EQU** X+5”是错误的

ABC **EQU** 2000H ；

ABC **EQU** 3000H ；错误，不能重复定义

STRING **EQU** <MOV AH,4CH> ；

3.3.4 伪指令

2. 数据定义伪指令

- 为数据项分配存储单元。汇编语言格式：

[变量名] 数据定义伪指令 操作数[, 操作数...]

- ✓ **变量名**：用户自定义标识符（即变量），表示操作数表首个数据项的逻辑地址，即用这个符号表示地址，常称为**符号地址**；
- ✓ 无变量名时，汇编程序直接为各操作数分配空间；
- 设置变量名方便存取指示存储单元。

助记符	变量类型	功能
DB	字节 (Byte)	给每个操作数项分配一个8位的字节单元；可用于定义字符串
DW	字 (word)	给每个操作数项分配一个16位的字单元；可用于定义数据、段地址、偏移地址
DD	双字 (Dword)	给每个操作数项分配一个32位的双字单元；可用于定义数据、含段地址和偏移地址的远指针
DF	3个字 (Fword)	给每个操作数项分配一个6字节单元；常用于表示32位CPU的48位远指针
DQ	4个字 (Qword)	给每个操作数项分配一个8字节单元；常用于表示64位数据
DT	10个字节 (Tbyte)	给每个操作数项分配一个10字节单元，常表示BCD码、数据（用于浮点运算）

3.3.4 伪指令

2. 数据定义伪指令

- **例3.1:** 有如下一些变量定义，右图是相应的内存分配示意图。

DATA1 DB -1*3

DATA2 DW 0204H,100H

DATA3 DD 12345H

STRING DB '0123','A','BC'

- 内存分配按照变量定义的先后依次分配

2000H:0000H	FDH	DATA1
2000H:0001H	04H	DATA2
	02H	
	00H	
	01H	
2000H:0005H	45H	DATA3
	23H	
	01H	
	00H	
2000H:0009H	30H	STRING
	31H	
	32H	
	33H	
	41H	
	42H	
	43H	

3.3.4 伪指令

2. 数据定义伪指令

DATA5 DW '01','23'

;如果用**DW**来定义字符串，则操作数项单引号内须是两个字符

DATA6 DB ?

;从DATA6开始**预留**一个字节单元

DATA7 DB 3 DUP(00)

;从DATA7开始分配3个字节单元，初值为0

DATA8 DW 2 DUP(?)

;从DATA8开始预留2个字

DATA9 DW DATA5

;将符号地址DATA5的值（偏移地址）存在DATA9开始的单元

DATA10 DW \$-DATA9

;\$表示当前地址指针（偏移地址）
;\$-DATA9，当前地址-DATA9偏移地址
;=DATA9数据区的长度（字节数）

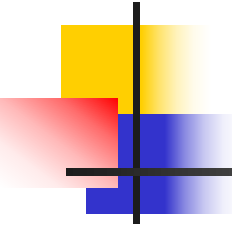
2000H:0010H	31H	DATA5
	30H	
	33H	
	32H	
2000H:0014H	××H	DATA6
2000H:0015H	00H	DATA7
	00H	
	00H	
2000H:0018H	××H	DATA8
	××H	
	××H	
	××H	
2000H:001CH	10H	DATA9
	00H	
2000H:001EH	02H	DATA10
	00H	

3.3.4 伪指令

3. 分析运算符和合成运算符

- 分析运算符可从标号或变量中分解出某些属性值

带分析运算符的表达式	表达式的值
OFFSET 变量名或标号	取出变量名或标号所在段的偏移地址
SEG 变量名或标号	取出变量名或标号所在段的段基地址
TYPE 变量名或标号	对字节型变量名返回值1，字型变量名返回值2，双字型变量名返回值4，三字型变量名返回值6，四字型变量名返回值8，十字节型变量名返回值10； 对标号进行操作，如果标号类型为NEAR，则返回-1；如果标号类型为FAR，则返回-2
LENGTH 变量名	取变量存储区中变量的元素个数（即变量定义中使用DUP的次数，无DUP时值为1）
SIZE 变量名	取变量存储区的总长度（以字节为单位），即 $SIZE=LENGTH \times TYPE$ 。



3.3.4 伪指令

3. 分析运算符和合成运算符

- 合成运算符可对标号或变量的类型属性进行设置

带合成运算符的表达式	表达式的作用
类型名 PTR 变量名或标号	将已定义的变量名或标号按该表达式指定类型名使用
SHORT 标号	将标号作为短处理
变量名或标号 EQU THIS	用于创建采用当前地址但为指定类型的操作数

3.3.4 伪指令

3. 分析运算符和合成运算符

例3.2 在例3.1的定义下，有下列语句

MOV DI, OFFSET DATA1 ;DI=0000H

MOV SI, OFFSET DATA1 ;SI=0000H

MOV AL, BYTE PTR [DI] ;AL=0FDH

MOV AX, WORD PTR [SI] ;AX=04FDH

MOV AX, SEG DATA1 ;AX=2000H

MOV DS, AX ;段基址送DS

MOV AL, LENGTH DATA2 ;AL=1

2000H:0000H	FDH	DATA1
2000H:0001H	04H	DATA2
	02H	
	00H	
	01H	
2000H:0005H	45H	DATA3
	23H	
	01H	
	00H	
2000H:0009H	30H	STRING
	31H	
	32H	
	33H	
	41H	
	42H	
	43H	

3.3.4 伪指令

3. 分析运算符和合成运算符

例3.2（续）：

MOV AL, LENGTH DATA7 ;AL=3

MOV AL, LENGTH DATA8 ;AL=2

MOV AL, SIZE DATA7 ;AL=3

MOV AL, SIZE DATA8 ;AL=4

2000H:0010H	30H	DATA5
	31H	
	32H	
	33H	
2000H:0014H	××H	DATA6
2000H:0015H	00H	DATA7
	00H	
	00H	
2000H:0018H	××H	DATA8
	××H	
	××H	
	××H	
2000H:001CH	10H	DATA9
	00H	

3.3.4 伪指令

4. 段定义伪指令--说明逻辑段的开始和结束

段名 **SEGMENT** [定位类型] [组合类型] ['类别名']

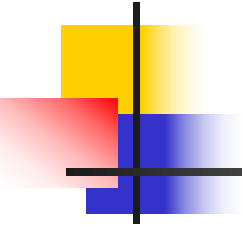
... ;本段语句序列（定义数据段、附加数据段和堆栈段时，只能包括伪指令语句；
... ;定义代码段时，可包括指令语句和与指令有关的伪指令语句。）

段名 **ENDS**

- ◆ **定位类型**：给出逻辑段在内存中实际地址边界。它有 **PAGE**、**PARA**、**WORD**、**BYTE** 四种类型。
- ◆ **组合类型**：在多模块程序设计中告诉汇编程序该段和其他同名段间如何组合连接。如果没有指明组合类型，则该程序段不与其他段相连接。有**PUBLIC**、**COMMON**、**AT**、**STACK**、**MEMORY**五种类型。
- ◆ **类别名**：是程序员任选的一个字符串，必须用单引号括起来。连接时，不同模块中有相同类别名的各段在物理地址上相邻地连接在一起，顺序则与 **LINK**时提供的各模块顺序一致。

3.3.4 伪指令——定位、组合类型

- **PAGE**: 段的起始地址能被256整除。此时该段 20 位物理地址为×××00H。
- **PARA**: 段的起始地址能被 16 整除。此时该段 20 位物理地址为××××0H。
- **WORD**: 段的起始地址能被 2 整除。此时该段 20 位物理地址为××××××××××××××0B。
- **BYTE**: 表示相应段可以从存储器的任何地址开始。
- **PUBLIC**: 表示该段与其他同名段在满足定位类型的前提下依次连接起来构成一个逻辑段。连接的顺序由连接程序 LINK 确定。
- **COMMON**: 表示该段与其他同名同类别段共享相同的存储空间。即各段都是从相同的地址开始，互相覆盖。连接后，段的长度等于最长的 COMMON 段的长度。
- **AT 表达式**: 表示相应段根据表达式的值定位段地址。例如，AT 2000H，表示该段的段基地址为 2000H，本段起始的物理地址 20000H。
- **STACK**: 与 PUBLIC 组合类型的处理方式相同，即把不同模块中带有 STACK 组合类型的同名段连接起来，使这些同名段都从同一基地址开始。但 STACK 组合方式仅用于堆栈段，并且，在可执行文件装入内存后，段寄存器内已是该段的段地址，堆栈指针 SP 已指向栈顶。
- **MEMORY**: 表示在连接时本段应装在同名段中具有最高地址的地方。若连接时具有 MEMORY 组合类型的段不止一个，则只有第一段才当成 MEMORY 组合类型来处理，其他的段将重叠，即按 COMMON 组合类型来处理。



3.3.4 伪指令

5. ORG伪指令

◆ 格式: **ORG 表达式**

◆ 规定其后的程序或数据块存放的起始地址，既以表达式的值为该起始地址（偏移地址）开始连续存放。

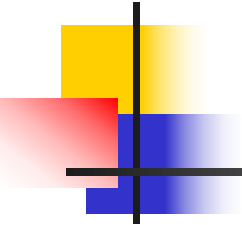
◆ ORG伪指令可以放置在源程序中的任何位置。

MY-SEG SEGMENT

ORG 1000H ;后面的数据区从偏移地址1000H开始
DATA1 DB 12H, 22, 34H

.....

MY-SEG ENDS



3.3.4 伪指令

6. ASSUME伪指令

- ◆ 格式: **ASSUME** 段寄存器名:段名[, 段寄存器名:段名, ...]
- ◆ 指示汇编程序使用哪个数据段、附加段、堆栈段、代码段, 即**指明段寄存器所对应的段**
- ◆ ASSUME语句**只能出现在代码段**内, 且一般是代码段内第一条语句; **允许多次使用**ASSUME语句, 重新约定段寄存器和段的关系
- ◆ **ASSUME** 语句并**不对相应的段寄存器赋值**。除了 **CS** 以外, 各个段寄存器的实际值, 还要用 **MOV** 指令来赋值。

MYDATA SEGMENT ;定义数据段 MYDATA

BUF DB 10 DUP(11H)

MYDATA ENDS

MYEXTRA SEGMENT ;定义附加数据段 MYEXTRA

EBUF DB 10 DUP(?)

MYEXTRA ENDS

MYSTACK SEGMENT ;定义堆栈段 MYSTACK

SBUF DB 200 DUP(0)

MYSTACK ENDS

MYCODE SEGMENT ;定义代码段 MYCODE

ASSUME CS:MYCODE, DS:MYDATA, ES:MYEXTRA

ASSUME SS:MYSTACK

START: MOV AX, MYDATA ; 段名具有段地址属性

MOV DS, AX ; 段基地址再送DS

MOV AX, MYEXTRA

MOV ES, AX

MOV AX, MYSTACK

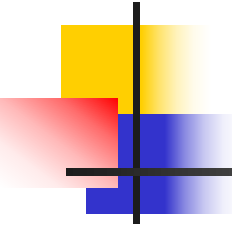
MOV SS, AX

MOV AH, 4CH

INT 21H

MYCODE ENDS

END START



3.3.4 伪指令

7. 过程定义伪指令

过程名 **PROC** [**NEAR/FAR**]

.....

RET ;至少有一条返回指令, 返回到调用程序

过程名 **ENDP**

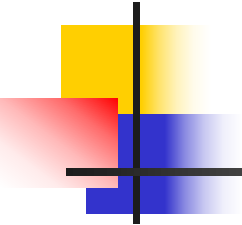
- ◆ 通常将具有某种功能的程序段定义成过程（即子程序）
- ◆ 其他程序用**CALL 过程名** 调用过程
- ◆ 过程至少有一条返回指令，以控制此过程执行完毕后返回到调用程序（主程序）
- ◆ **近过程**（与调用程序在同一个代码段），类型为**NEAR**，可以省略；
- ◆ **远过程**（与调用程序不在同一个代码段），类型为**FAR**，不能省略。

3.3.4 伪指令

7. 过程定义伪指令

例 3.3 编写一个 10ms 延时子程序。

```
DELY PROC           ;定义一个近过程
    PUSH BX          ;保护 BX 原来内容
    PUSH CX          ;保护 CX 原来内容
    MOV BL, 2         ;外循环次数
NEXT: MOV CX, 4167 ;内循环次数（实现延时 5ms）
Y10MS: LOOP Y10MS ;CX≠0 则循环
    DEC BL           ;修改外循环计数值
    JNZ NEXT         ;BL≠0 则进行第 2 轮循环
    POP CX           ;恢复 CX 原来内容
    POP BX           ;恢复 BX 原来内容
    RET              ;过程返回
DELY ENDP          ;过程结束
```



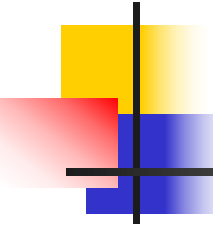
3.3.4 伪指令

8. 源程序结束伪指令**END**

◆ 伪指令**END**用来表明源程序到此处结束。格式：

END [表达式]

◆ 这里的表达式通常就是程序**第一条可执行指令语句的标号**。该标号就是程序在汇编、连接后，将目标代码装入内存**要执行的起始地址**。



3.4 指令系统

◆ 本节主要介绍 **80X86** 的常用指令——整数指令

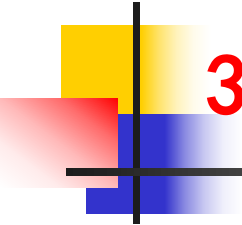
◆ 整数指令按功能分：

- ◆ 数据传送类指令
- ◆ 算术运算类指令
- ◆ 逻辑运算与移位指令
- ◆ 串操作指令
- ◆ 位操作指令
- ◆ 控制转移指令
- ◆ 标志操作指令
- ◆ CPU控制指令
- ◆ 高级语言指令等

3.4 指令系统

◆ 指令符号说明

符 号	意义说明
OP1, OP2	操作数。多操作数指令中，OP1为目的操作数，OP2为源操作数
reg	通用寄存器。可以是8位、16位或32位
sreg	段寄存器：CS, DS, ES, SS, FS, GS
reg8	8位通用寄存器：AH, AL, BH, BL, CH, CL, DH, DL
reg16	16位通用寄存器：AX, BX, CX, DX, DI, SI, SP, BP
reg32	32位通用寄存器：EAX, EBX, ECX, EDX, EDI, ESI, ESP, EBP
mem	存储器。可以是8位、16位或32位
mem8	8位存储器。相应操作数只能是8位
mem16	16位存储器。相应操作数只能是16位
mem32	32位存储器。相应操作数只能是32位
imm	立即数。可以是8位、16位或32位
imm8	8位立即数。有关操作数只能是8位



3.4.1 数据传送类指令

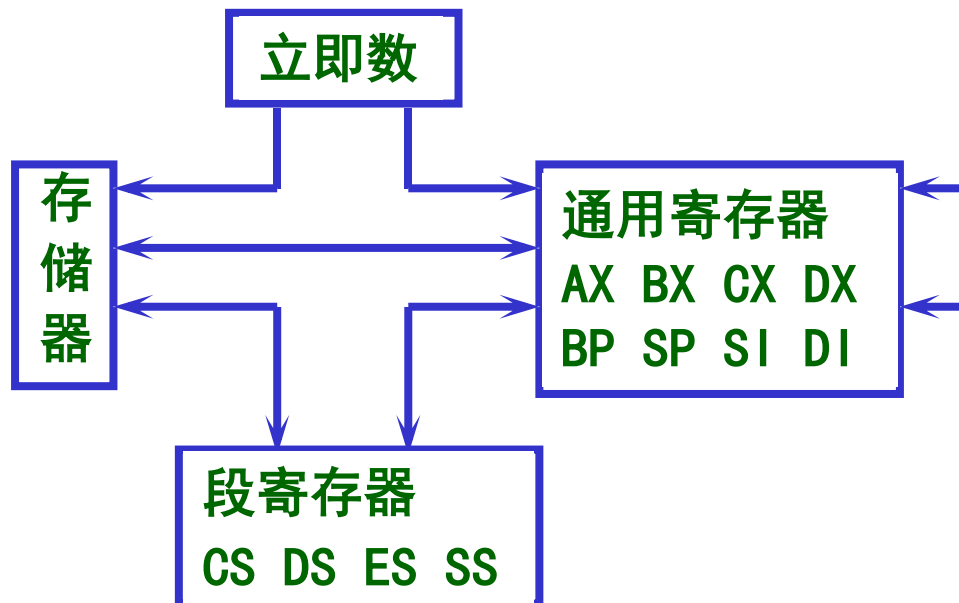
- ◆ 除标志传送指令外，指令执行的结果不影响标志寄存器中的标志位。
- 这类指令分为分为
 - 通用数据传送指令
 - 交换指令
 - 堆栈操作指令
 - 地址传送指令
 - 输入/输出指令

3.4.1 数据传送类指令

(1) 通用传送指令MOV

指令格式: **MOV OP1, OP2**

- **MOV sreg, reg/mem**
- MOV reg, reg/sreg/mem/imm**
- MOV mem, reg/sreg/imm**



注意:

- MOV不影响标志位
- 目的操作数不能为 CS、IP (EIP) 和立即数。
- 源和目的操作数必须等长
- 源与目的操作数不能同时为存储器操作数或段寄存器
- 立即数不能传送段寄存器

例如: ~~MOV ES, DS~~
~~MOV DS, 2000H~~
~~MOV CS, AX~~
~~MOV [BX], [SI]~~

3.4.1 数据传送类指令

(2) 交换指令

◆ 交换指令格式: **XCHG OP1, OP2**

• **XCHG reg, reg/mem**

注意:

- 两操作数不允许都是存储器
- 源和目的操作数必须等长
- 两操作数中不允许有立即数
- 两操作数中不允许有段寄存器

例如: XCHG BX, [DI+2400]
XCHG AH, AL
XCHG CX, 100H; 错误

3.4.1 数据传送类指令

(3) 地址传送指令

◆ 有效地址 (EA) 传送指令LEA格式: **LEA OP1, OP2**

• **LEA reg16, mem**

例如: **LEA** BX, TABLE ;将变量 TABLE 的偏移地址送 BX。
;等同于 MOV BX, **OFFSET** TABLE

MOV BX, 400H

MOV SI, 3CH

LEA BX, [BX+SI+0F62H] ;BX←400H+3CH+0F62H=139EH

3.4.1 数据传送类指令

(4) 堆栈操作指令

- 堆栈是一个“后进先出LIFO”（或说“先进后出FILO”）的主存区域，位于堆栈段中；SS段寄存器记录其段地址
- 堆栈只有一个出口，即当前栈顶；用堆栈指针寄存器SP指定
- 堆栈只有两种基本操作：进栈和出栈，对应两条指令**PUSH**和**POP**

进栈指令：**PUSH r16/m16/seg**

； $SP \leftarrow SP - 2$

； $SS:[SP] \leftarrow r16/m16/seg \text{ 低 } 8 \text{ 位}$

； $SS:[SP+1] \leftarrow r16/m16/seg \text{ 高 } 8 \text{ 位}$

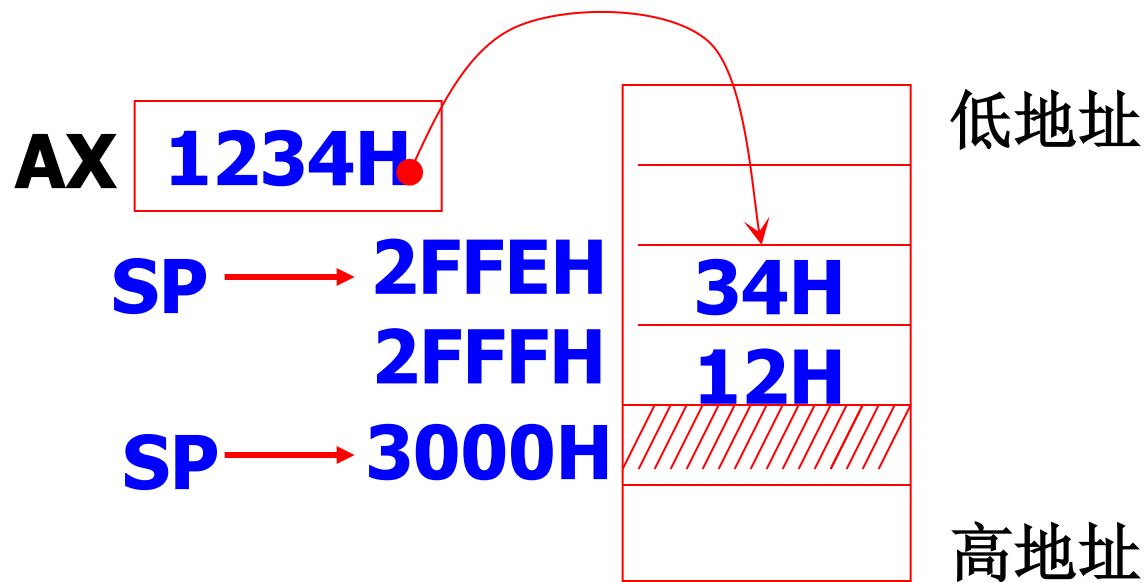
出栈指令：**POP r16/m16/seg**

； $r16/m16/seg \text{ 低 } 8 \text{ 位} \leftarrow SS:[SP]$

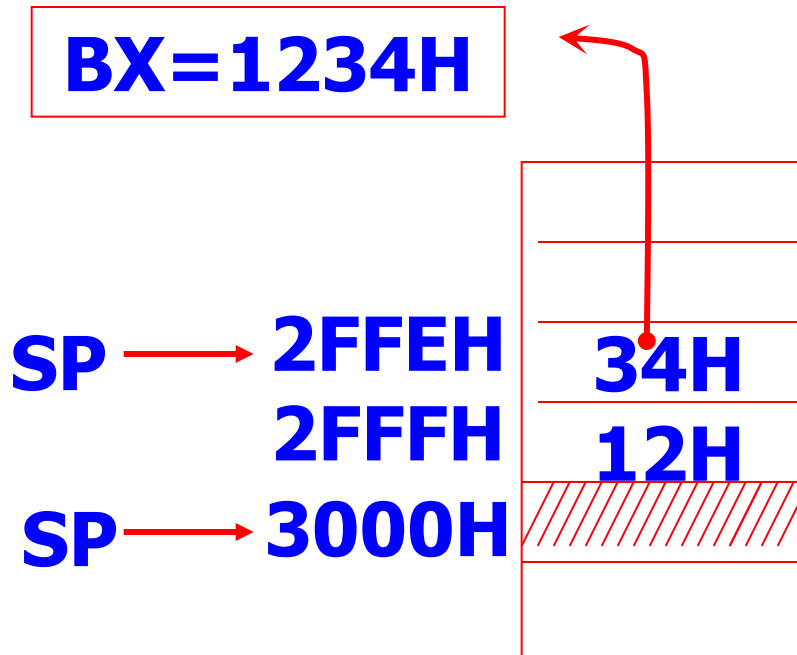
； $r16/m16/seg \text{ 高 } 8 \text{ 位} \leftarrow SS:[SP+1]$

； $SP \leftarrow SP + 2$

PUSH AX



POP BX



3.4.1 数据传送类指令

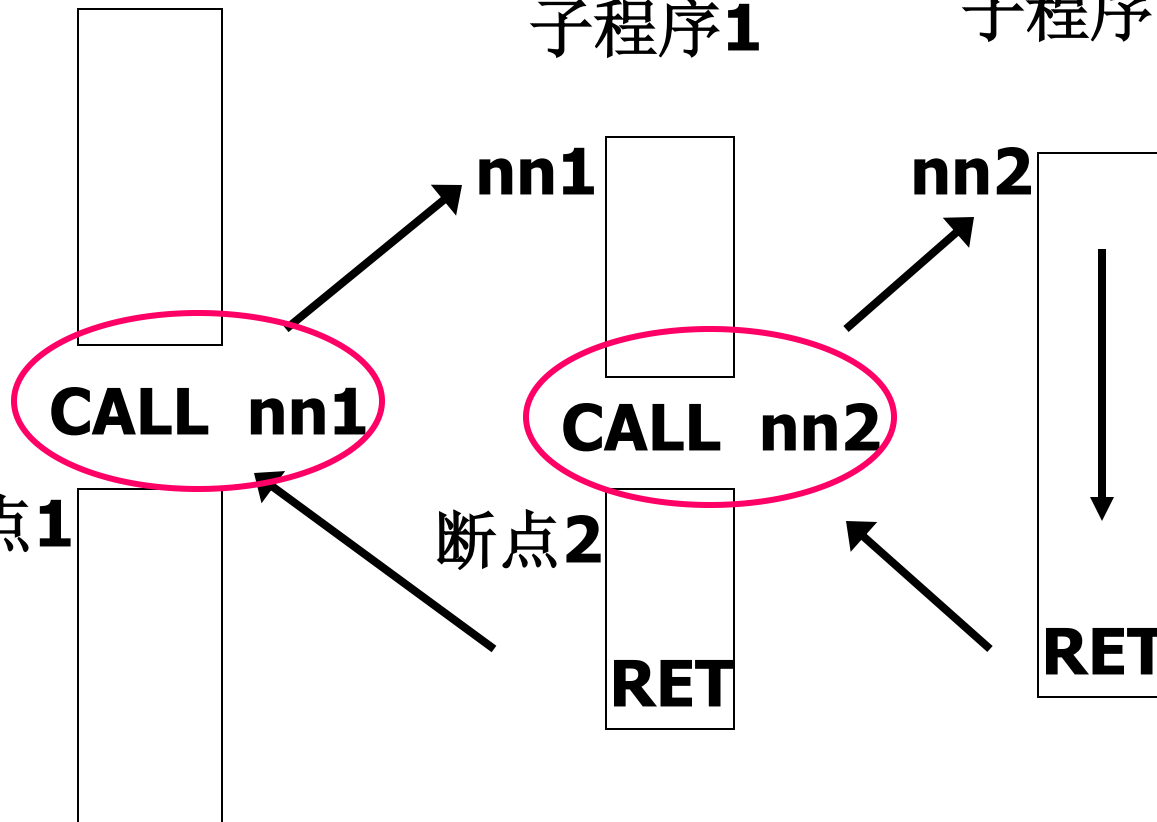
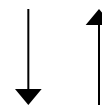
CS:IP

主程序段

子程序1

子程序2

栈顶



栈顶

断点2

栈顶

断点1

栈顶

栈底

内存堆栈区

堆栈：主要用于子程序调用、中断响应等操作时的参数保护，也可用于实现参数传递。

3.4.1 数据传送类指令

(5) 输入输出 (I/O) 指令

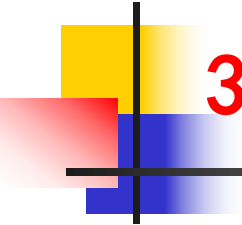
- ◆ 指令格式: **IN** AL/AX, imm8/DX ; 输入指令
OUT imm8/DX, AL/AX ; 输出指令
- ◆ IN 用于从 I/O 端口读数据到累加器 AL 或 AX 中
- ◆ OUT 用于把累加器 AL 或 AX 的内容写到 I/O 端口
- ◆ I/O 端口寻址方式: 直接寻址和 **DX** 寄存器间接寻址
- ◆ 直接寻址: 当 **I/O** 端口地址可用 **8** 位立即数表示时可寻址的 **I/O** 端口地址范围是 **00~FFH**。

IN AL, 3FH

IN AX, 3FH ;从 3FH、40H 的 2 个端口读入一个字到 AX

OUT 43H, AL

OUT 43H, AX ;把 AX 的字数据写入 43H、44H 端口



3.4.1 数据传送类指令

(5) 输入输出 (I/O) 指令

◆ **DX** 寄存器间接寻址：当 **I/O** 端口地址为 **16** 位时，由 **DX** 寄存器指定端口号，可寻址的 **I/O** 端口地址范围是 **0000~FFFFH**。

MOV DX, 03B0H

IN AL, DX

IN AX, DX

OUT DX, AL

OUT DX, AX

加1/减1指令

◆ 加1指令 **INC reg/mem (8/16)**

例: **INC BX ; BX ← BX+1**

说明: 无符号数加1; 不影响进位标志

INC BYTE PTR [2000H]

INC WORD PTR [2000H]

1000H: 2000H

00H

00H

1000H: 2000H

00H

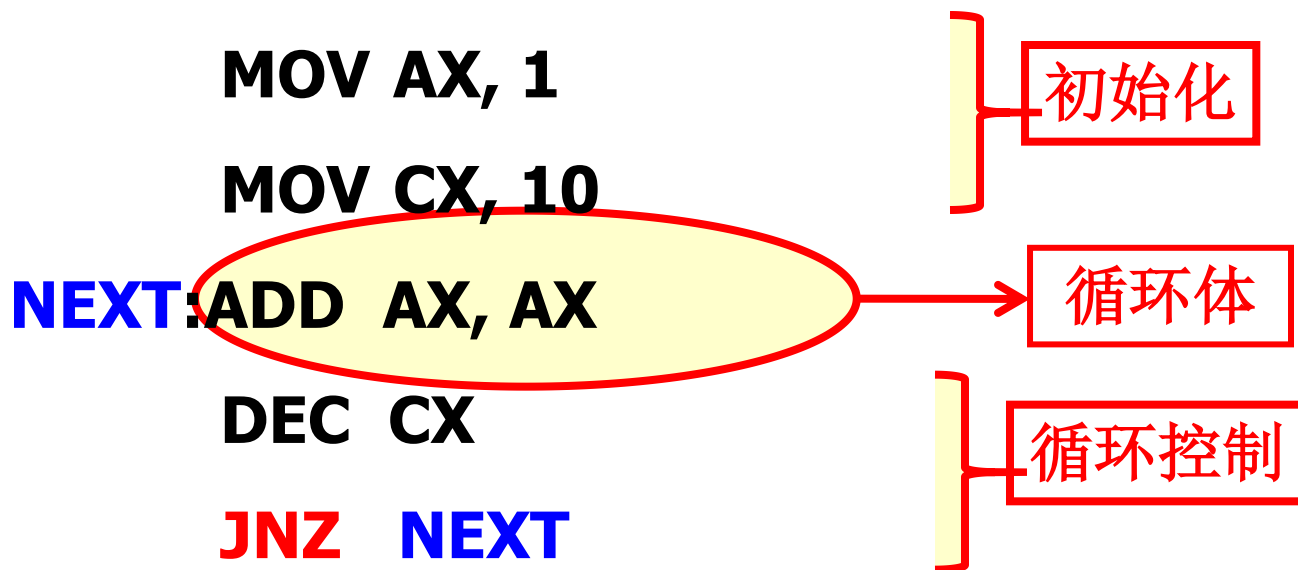
01H

◆ 减1指令 **DEC DEC reg/mem (8/16)**

说明: 无符号数减1; 不影响借位标志

条件转移指令 (**JNZ/JNE** 标号)

- ◆ 结果不为零 (即 **ZF=0**) 转移到标号处继续执行;
标号指向指令与本指令距离在 **-128~127** 之间



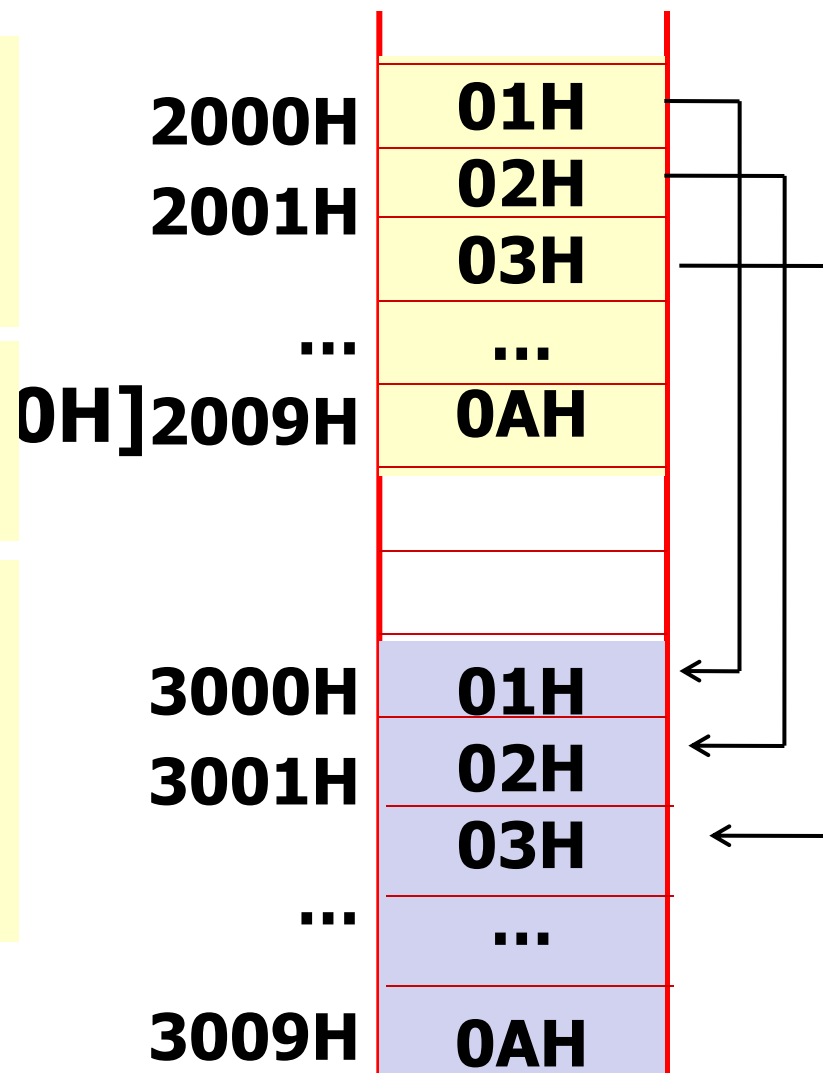
习题1-1:

将数据段中**2000H**单元开始存放的**10**个数（**8**位二进制数）传送到**3000H**开始的**10**个字节中

```
MOV SI, 2000H
MOV DI, 3000H
MOV CX, 10
```

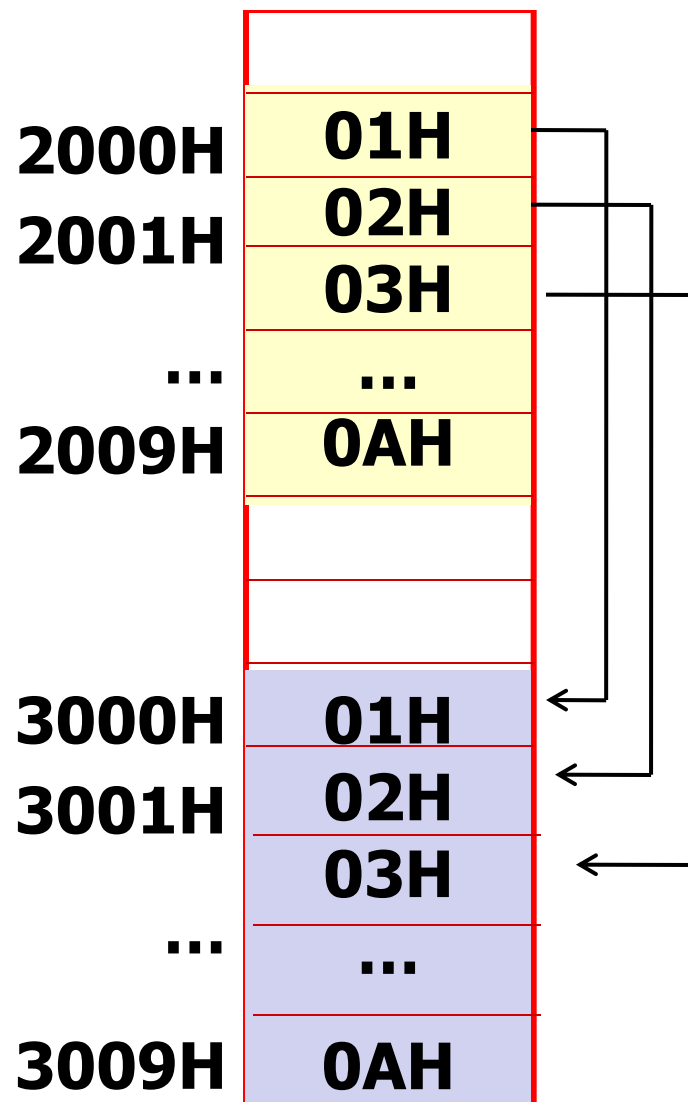
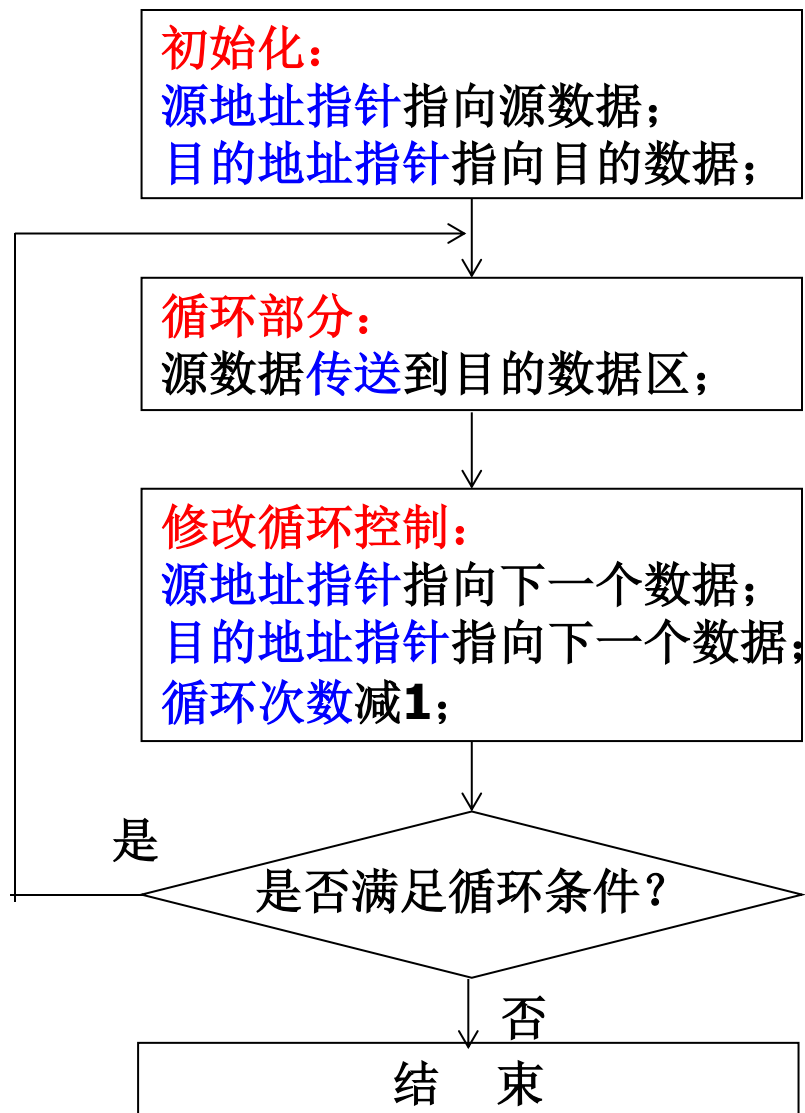
```
NEXT:  MOV AL, [SI]
        MOV [DI], AL
```

```
INC SI
INC DI
DEC CX
JNZ NEXT
```



习题1-1:

将数据段中**2000H**单元开始存放的**10**个数（**8**位二进制数）传送到**3000H**开始的**10**个字节中



习题1-1:

将数据段中**2000H**单元开始存放的**10**个数（**8**位二进制数）传送到**3000H**开始的**10**个字节中

```
ORG 2000H
```

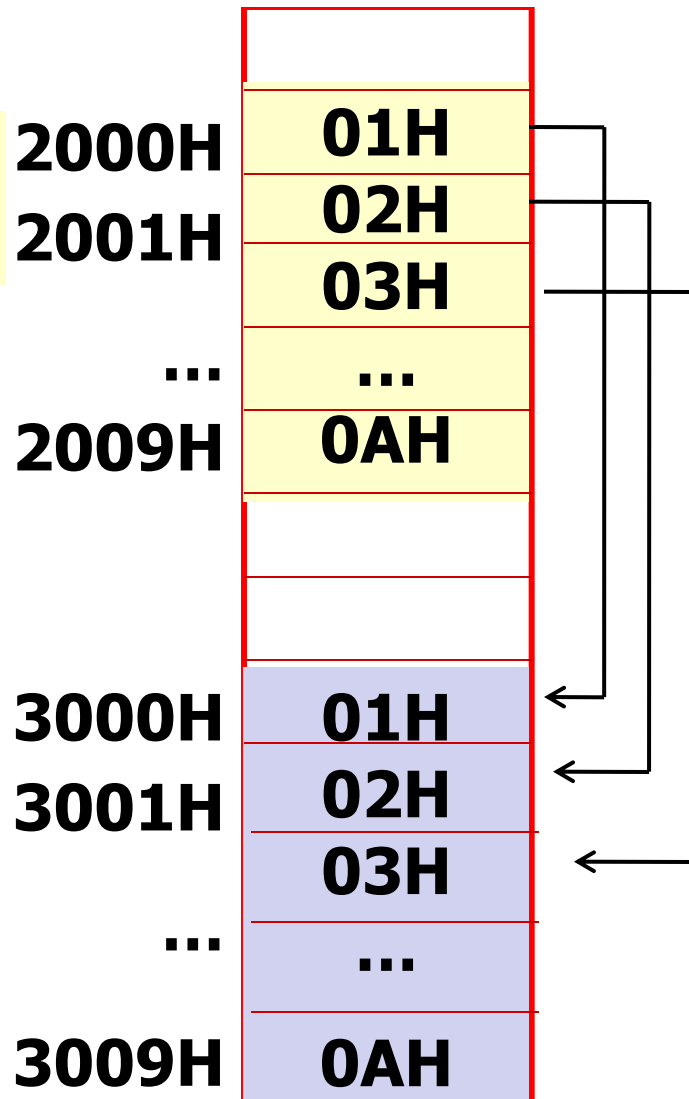
```
DAT1 DB 01H,02H,03H,04H,05H 2000H  
      DB 06H,07H,08H,09H,0AH 2001H
```

```
ORG 3000H
```

```
DAT2 DB 10 DUP (0) 2009H
```

```
MOV SI, 2000H  
MOV DI, 3000H
```

```
LEA SI, DAT1  
LEA DI, DAT2
```



截图保存: (1) 源程序

```
01 DATAS SEGMENT
02     ;此处输入数据段代码
03     ORG 2000H
04     DAT1 DB 1,2,3,4,5,6,7,8,9,10
05     ORG 3000H
06     DAT2 DB 10 DUP(0)
07 DATAS ENDS
08
09 STACKS SEGMENT
10     ;此处输入堆栈段代码
11 STACKS ENDS
12
13 CODES SEGMENT
14     ASSUME CS:CODES,DS:DATAS,SS:STACKS
15 START:
16     MOV AX,DATAS
17     MOV DS,AX
18     ;此处输入代码段代码
19     LEA SI, DAT1
20     LEA DI, DAT2
21     MOV CX, 10
22 NEXT:MOV AL,[SI]
23     MOV [DI],AL
24     INC SI
25     INC DI
26     DEC CX
27     JNZ NEXT
28     MOV AH,4CH
29     INT 21H
30 CODES ENDS
31     END START
```

(2) 反汇编程序

```
-U
16F3:0000 B8F213      MOV     AX,13F2
16F3:0003 8ED8             MOV     DS,AX
16F3:0005 8D360020        LEA     SI,[2000]
16F3:0009 8D3E0030        LEA     DI,[3000]
16F3:000D B90A00        MOV     CX,000A
16F3:0010 8A04             MOV     AL,[SI]
16F3:0012 8805             MOV     [DI],AL
16F3:0014 46              INC     SI
16F3:0015 47              INC     DI
16F3:0016 49              DEC     CX
16F3:0017 75F7             JNZ     0010
16F3:0019 B44C             MOV     AH,4C
16F3:001B CD21      INT     21
16F3:001D 0000             ADD     [BX+SI],AL
16F3:001F 0000             ADD     [BX+SI],AL
-
```

(3) 程序运行过程

```
-G=0000 0019
AX=130A BX=0000 CX=0000 DX=0000 SP=0000 BP=0000 SI=200A DI=300A
DS=13F2 ES=13E2 SS=13F2 CS=16F3 IP=0019  NV UP EI PL ZR NA PE NC
16F3:0019 B44C             MOV     AH,4C
```


MASM2015调试

(4) 查看结果 (运行后)

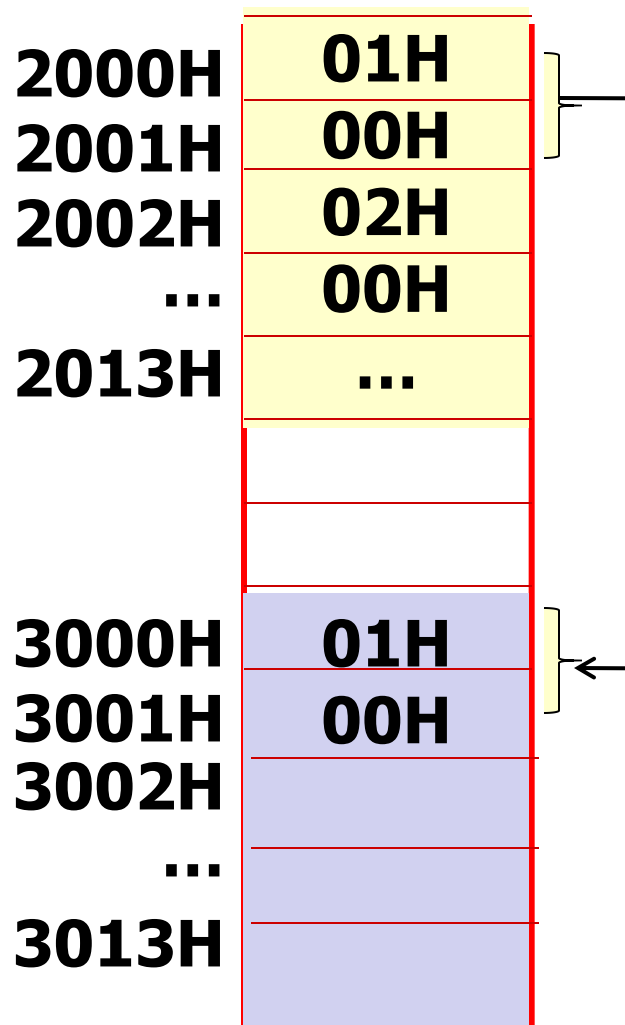
```
-D 13F2:2000
13F2:2000  01 02 03 04 05 06 07 08-09 0A 00 00 00 00 00 00 00 .....
13F2:2010  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
13F2:2020  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
13F2:2030  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
13F2:2040  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
13F2:2050  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
13F2:2060  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
13F2:2070  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
-D 13F2:3000
13F2:3000  01 02 03 04 05 06 07 08-09 0A 00 00 00 00 00 00 .....
13F2:3010  B8 F2 13 8E D8 8D 36 00-20 8D 3E 00 30 B9 0A 00 .....6. >.0...
13F2:3020  8A 04 88 05 46 47 49 75-F7 B4 4C CD 21 00 00 00 ....FGIu..L.?!...
13F2:3030  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
13F2:3040  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
13F2:3050  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
13F2:3060  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
13F2:3070  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
```

习题1-2:

将数据段**DATA1**的**10**个字数据（**16**位数）传送到附加段**DATA2**中。

```
MOV AL, [SI]
MOV [DI], AL; 字节传送
INC SI
INC DI
```

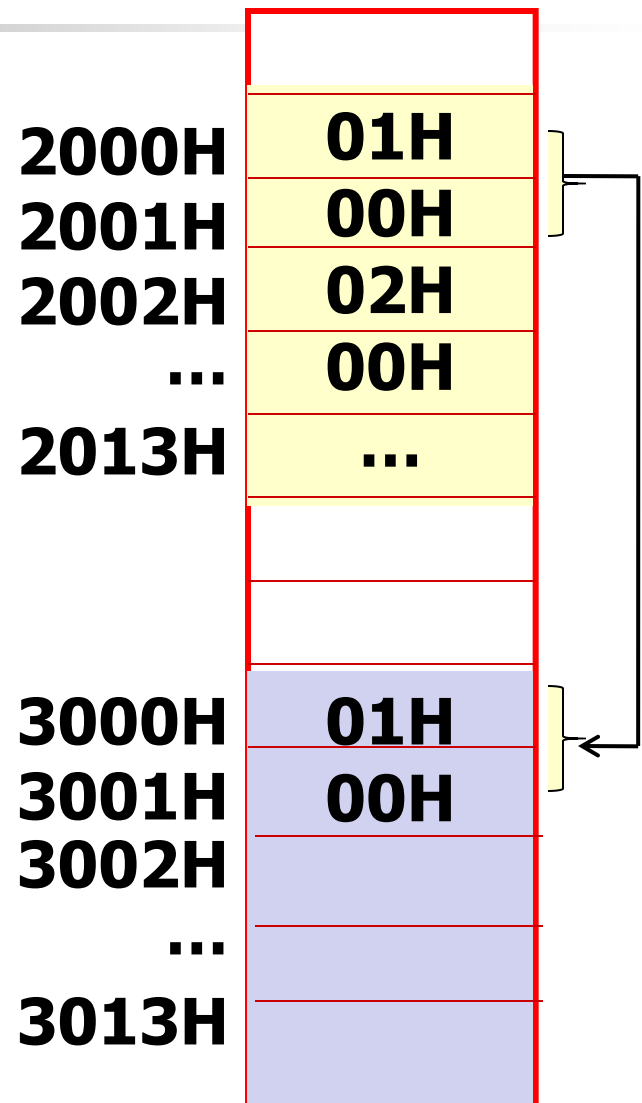
```
MOV AX, [SI]; 0001H
MOV [DI], AX; 字传送
INC SI
INC SI
INC DI
INC DI
```



```

01 DATA1 SEGMENT
02 D1 DW 1,2,3,4,5,6,7,8,9,10
03 DATA1 ENDS
04
05 DATA2 SEGMENT
06 D2 DW 10 DUP(0)
07 DATA2 ENDS
08
09 CODES SEGMENT
10     ASSUME CS:CODES, DS:DATA1, ES:DATA2
11 START:
12     MOV AX, DATA1
13     MOV DS, AX
14     MOV AX, DATA2
15     MOV ES, AX
16     ;此处输入代码段代码
17     LEA SI, D1
18     LEA DI, D2
19     MOV CX, 10
20 NEXT: MOV AX, [SI]
21     MOV ES:[DI], AX
22     INC SI
23     INC SI
24     INC DI
25     INC DI
26     DEC CX
27     JNZ NEXT
28     MOV AH, 4CH
29     INT 21H
30 CODES ENDS
31 END START

```



(6) XLAT 查表指令:

• 格式: **XLAT** ; **AL** ← **DS:[BX+AL]**

或 **XLAT OPR** ; **OPR** 表示转换表的首地址

```
LEA  BX, TABLE
```

```
MOV  AL, 3
```

```
MOV  AH, 0
```

```
ADD  BX, AX
```

```
MOV  AL, [BX]
```

XLAT

BX → TABLE

AL →

AL

9

0
1
4
9
16

N

0

1

2

3

4

注意: 换码表的长度不能超过**256**个字节

习题2-1、已知**0~15**的平方表，查表求**X**（**X**是**0**到**15**间任一数）的平方，并送到**Y**单元，用两种方法。

方法1:

```
LEA    BX,    PFTABLE
MOV     AL,    X
MOV     AH,    0
ADD     BX,    AX
MOV     AL,    [BX]
MOV     Y,     AL
```

方法2:

```
LEA    BX,    PFTABLE
MOV     AL,    X
XLAT
MOV     Y,     AL
```

PFTABLE	+0	00H	0
	+1	01H	1
	+2	04H	4
	+3	09H	9
	+4	10H	16
	+5	19H	25
	+6	...	
	+15	E1H	225

习题2-2、已知**0~255**的平方表，查表求**X**的平方，并送到**Y**单元（可只定义**16**个或编程写入全部数据）。

注意：一个平方值占了两个字节，故**X**的平方的地址相对首地址的偏移量为**2X**。

```
LEA    BX,    PFTABLE
MOV    AL,    X
MOV    AH,    0
ADD    BX,    AX
ADD    BX,    AX
MOV    AX,    [BX]
MOV    Y,    AX
```

PFTABLE			
BX →	+0	00H	0
	+1	00H	
	→+2	01H	1
	+3	00H	
	→+4	04H	2
	+5	00H	
	→+6	09H	3
	+7	00H	
	→+8	10H	4
	+9	00H	
		...	
+...		255	

70